Database Schema

Management approach

Mechanism	Role
Hibernate `ddl-auto=update`	Creates/updates most tables from entities
`schema.sql`	Creates `account_ordinal_seq`
Seed runners	`RoleInitializer`, `AdminRoleInitializer`, `AccountPolicyInitializer`

Entity-relationship (current)

```
erDiagram
  customers ||--o{ account : has
  customers ||--o{ beneficiary : saves
  customers ||--o{ transfer_request : requests
  users ||--o{ user_roles : has
  roles ||--o{ user_roles : grants
  account ||--o{ bank_transaction : posts
  account_policy {
    bigint policy_id PK
    string account_type
    string currency_code
  }
  customers {
    bigint customer_id PK
    decimal transfer_approval_threshold
  }
  account {
    bigint account_id PK
    string account_number UK
    bigint customer_id FK
  }
  users {
    bigint user_id PK
    string username UK
    bigint customer_id
  }
  beneficiary {
    bigint beneficiary_id PK
    bigint customer_id FK
    string bank_type
  }
  transfer_request {
    bigint transfer_request_id PK
    bigint customer_id FK
    string status
  }
  audit_event {
    bigint id PK
    string category
    string action
  }
  bank_transaction {
    bigint transaction_id PK
    bigint account_id FK
  }
```

Tables

`customers`

Entity: `com.bankone.customer.entity.Customer`

Column	Notes
`customer_id`	PK, identity
`first_name`, `last_name`	Required
`email`, `phone_number`	Unique
`date_of_birth`	Optional
`address`, `status`	Required
`transfer_approval_threshold`	Optional; portal same-bank gate
`created_at`, `updated_at`	`@PrePersist` / `@PreUpdate`

`customerCode` is **not** stored; JSON accessor formats ID via `BusinessIdFormatter`.

`account`

Entity: `com.bankone.account.entity.Account`

Column	Notes
`account_id`	PK
`account_number`	Unique, generated
`branch_code`	String (no branch table)
`account_type`, `currency_code`, `ordinal`, `check_digit`	Number composition
`available_balance`, `ledger_balance`	Updated on open/deposit
`debit_count`, `credit_count`, `last_*_at`	Counters / stamps
`status`	ACTIVE, FROZEN, DORMANT, SUSPENDED, CLOSED
`customer_id`	FK → `customers`
Audit-ish	`created_at`, `activated_at`, `closed_at`, `created_by`, `closed_by`

`account_policy`

Entity: `com.bankone.account.entity.AccountPolicy` extends `AuditableEntity`

Unique `(account_type, currency_code)`.

Seeded INR policies for CURRENT, SAVINGS, SALARY, FIXED_DEPOSIT, RECURRING_DEPOSIT. **No LOAN seed.**

`users`, `roles`, `user_roles`, `role_access`

Entities under `com.bankone.user` / `com.bankone.role`. Sequences: `user_seq`, `role_seq`, `user_role_seq`.

`users.customer_id` — null for staff; set for portal logins.

`role_access` — element collection of access codes (`AppAccess`).

`AuditableEntity` columns on User/Role/UserRole/AccountPolicy: `created_at`, `updated_at`, `created_by`, `updated_by`, `version`.

`bank_transaction`

Entity: `com.bankone.transaction.entity.Transaction`

Table name is `bank_transaction` (avoids SQL keyword `transaction`).

Column	Notes
`transaction_id`	PK (identity)
`account_id`	FK → `account` (required)
`transaction_type`	Enum string: CREDIT, DEBIT
`amount`	> 0; precision 19,2
`balance_after`	Account ledger balance after posting
`currency_code`	Copied from account (length 3)
`narration`	Optional text
`created_at`	Set in `@PrePersist`
`created_by`	Staff username when provided

Written from open/deposit/withdraw/transfer via `TransactionService.record`. Staff list: `GET /transactions`.

`beneficiary`

Portal payees (`SAME_BANK` / `OTHER_BANK`), soft-deactivate via `active=false`.

`transfer_request`

Portal transfers awaiting or after staff resolution (`PENDING`, `APPROVED`, `REJECTED`, `EXECUTED`).

`audit_event`

Activity trail — see MODULES/Audit.md.

Sequences

```
-- schema.sql
CREATE SEQUENCE IF NOT EXISTS account_ordinal_seq
START WITH 1 INCREMENT BY 1;
```

Used by `AccountRepository.getNextOrdinal()`. Also Hibernate sequences for users/roles/transfer_request/beneficiary.

Not present (planned)

No tables yet for: `branch` master, `loan` product, outbox (learning plan), shard metadata tables.

Note: Rate-limit and entity **cache** use **Redis** (not Postgres tables). Read-replica lab uses a second database `bankone_read`. See MODULES/Caching.md and ARCHITECTURE.

Migration notes

Changing entity fields with `ddl-auto=update` alters tables in place. For production-ready history, replace with Flyway/Liquibase (see EXTENSION_GUIDE and TECH_LEARNING_PLAN).